

客户服务器模式 和套接字API

理论课程

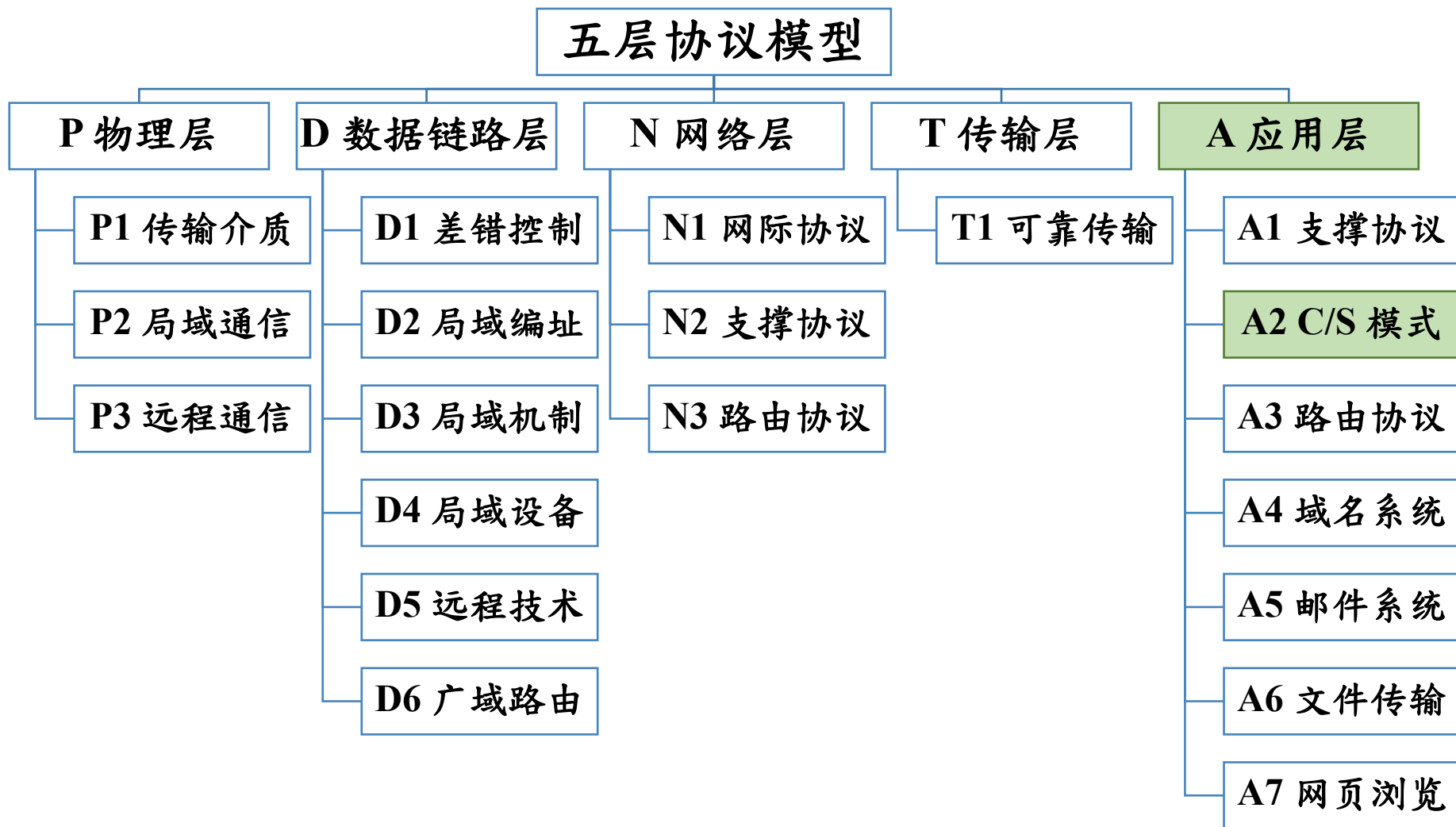


廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

知识框架



主要内容

- 客户端—服务器端（C/S）交互模式工作原理
- 并发的概念
- Socket结构、半相关与全相关
- 服务器与用户、服务器端与客户端，二者区别
- Socket API主要函数（Windows或Linux系统下）
- 流模式的客户端、服务器端Socket API调用流程
- 报文模式的客户端、服务器端Socket API调用流程

对应课本章节

- **PART I Introduction And Internet Applications**
 - **Chapter 3 Internet Applications And Network Programming**

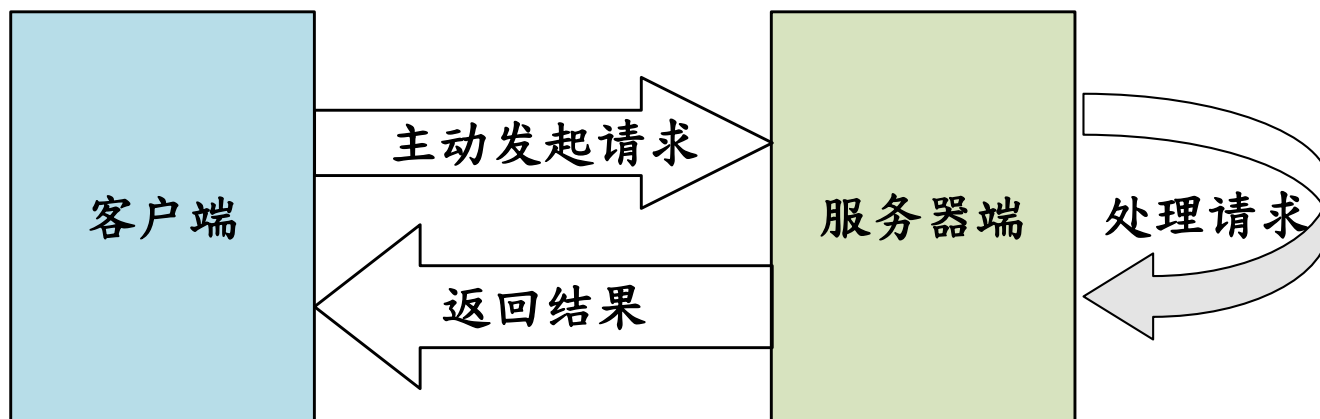


内容纲要

1	客户端和服务端
2	网间进程通信
3	Socket API
4	客户服务器端程序实例
5	小结

客户和服务 器

- 从传统单机程序，到让多个用户共享资源和服务
- 将计算任务分为服务的请求方和提供方两部分。
 - 服务器端（Server）：被动等待来自客户的请求，处理请求并传回结果。
 - 客户端（Client）：主动发送请求，等待直到收到响应。

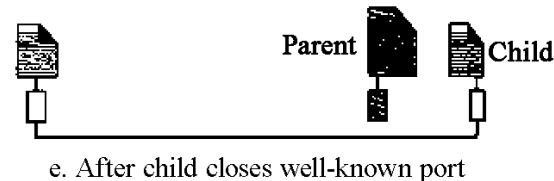
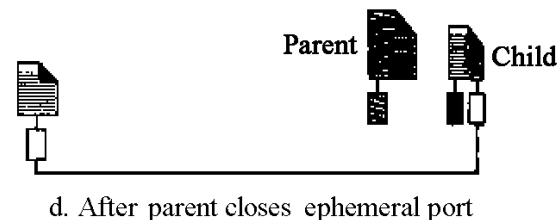
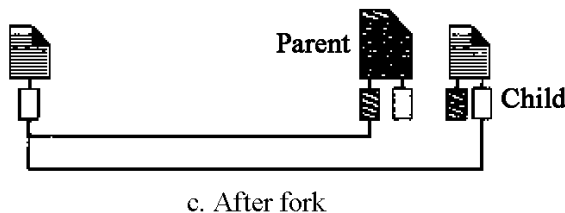
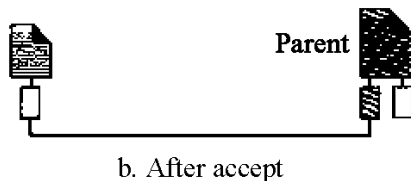
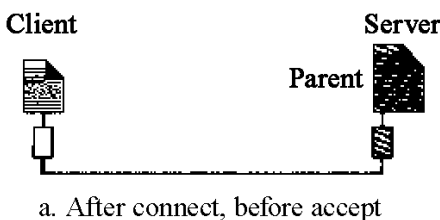


客户端或服务端

- 客户端或服务端，可以用来指程序、软件或硬件。
 - 一个软件系统由多个程序和数据共同完成功能，通常包含客户端程序和服务器端程序。
- 一个硬件可以运行多个软件，包括客户和服务软件。
 - 服务器硬件主要运行服务器软件，也可以运行客户端软件
 - 服务器软件主要调用服务器程序，也可以调用客户端程序
 - 客户端硬件主要运行客户端软件，也可以运行客户端软件
 - 客户端软件主要调用客户端程序，也可以调用客户端程序

并发 (Concurrency)

- 并发是指系统能够同时处理多个任务的能力
 - 服务器无需等待前一个客户端完成操作，即可响应新请求。
 - 允许多个应用同时执行的计算机系统称为支持并发的系统。
- 并发模式
 - 多进程
 - 多线程



内容纲要

1	客户端和服务端
2	网间进程通信
3	Socket API
4	客户服务器端程序实例
5	小结

进程通信

- 进程通信的概念最初来源于单机系统。
 - 操作系统提供了管道、命名管道和软中断信号，消息、共享存储区和信号量等，仅限于用在本机进程之间通信。
- 网间进程通信要解决不同主机进程间的相互通信问题。
 - 首先要解决的是网间进程标识问题。同一主机上，不同进程可用进程号（process ID）唯一标识。但在网络环境下，各主机独立分配的进程号不能唯一标识该进程。
 - 其次，操作系统支持的网络协议众多，不同协议的工作方式不同，地址格式也不同。因此，网间进程通信还要解决多重协议的识别问题。

标识网间通信的进程

- IP地址标识一台主机，但无法区分主机上的程序
 - 一台主机上的计算机系统可以同时运行多个网络程序
- 端口号标识在同一台主机上不同的网络应用程序
- 完整的通信地址：IP地址+端口号
- 协议类型
 - TCP和UDP程序分别通过端口地址，确定发送主机希望将数据报交付给目标主机上的哪个应用层程序。

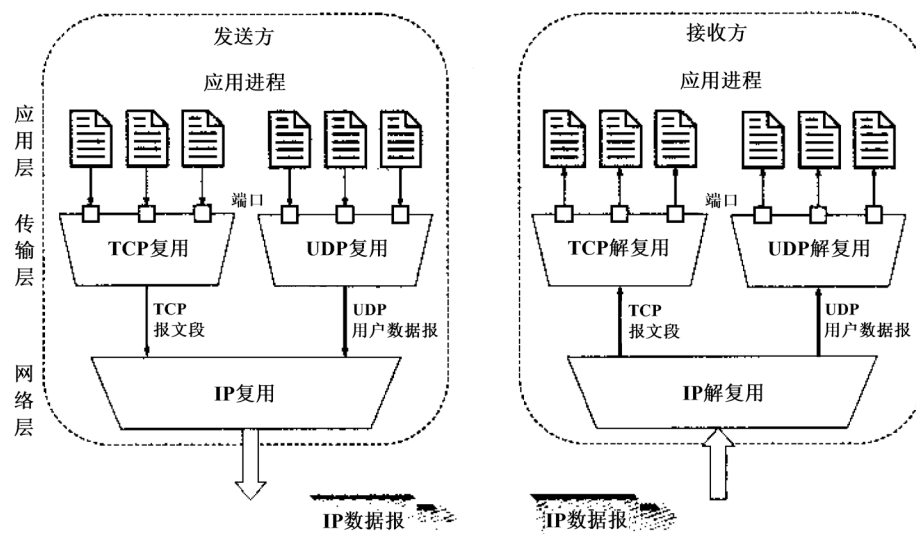


图 7-7 端口在进程之间的通信中所起的作用

协议端口

- 协议端口（ Protocol Port，简称端口）
 - 端口是一种抽象的软件结构（包括数据结构和I/O缓冲区）。
- 端口号（ Port Number）区别不同端口。
 - 取值：0～65535，分为公开端口、注册端口、动态端口
 - TCP和UDP的端口相互独立
- 端口间操作类似一般I/O操作
 - 进程获取端口，相当于获取本地I/O文件
 - 可用一般读写原语访问：开、关、读、写。

UDP报文队列

- UDP通过端口号将数据报定向至对应的应用层进程。
- 操作系统为每个UDP套接字维护两个独立的缓冲队列
 - 接收队列：存储从网络到达但尚未被应用层读取的数据报
 - 发送队列：暂存应用层提交但尚未发送到网络的数据报

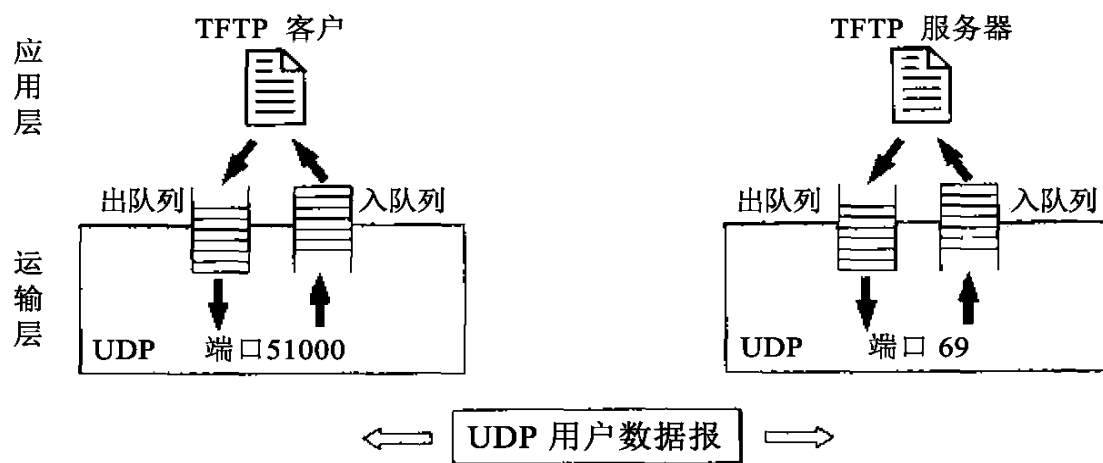
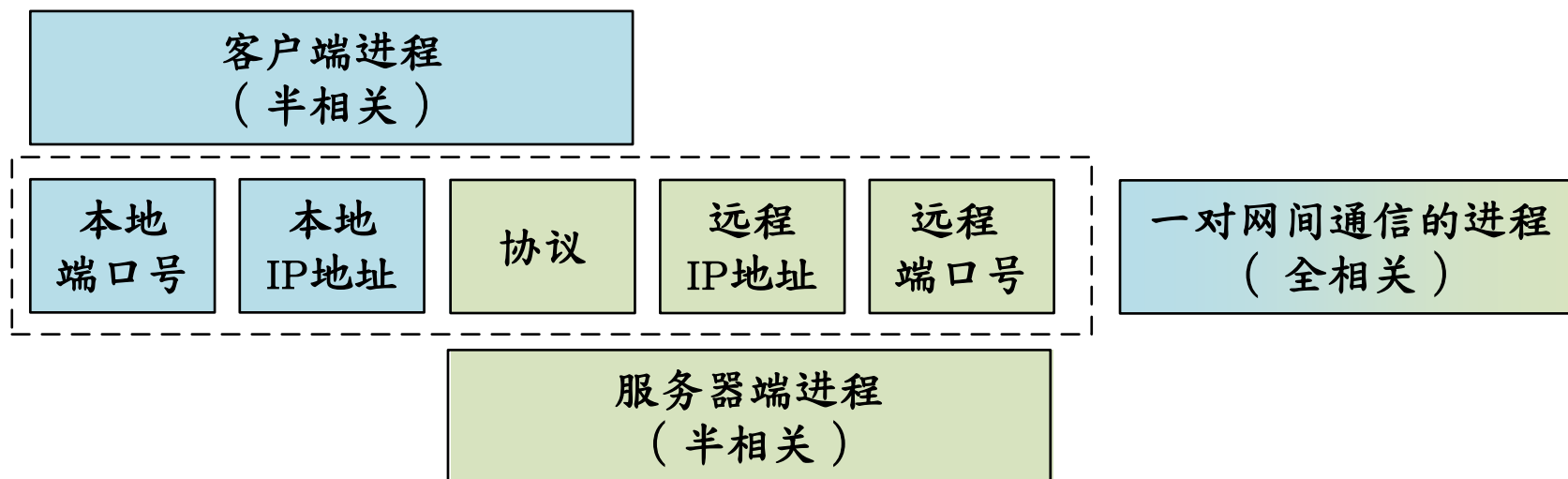


图7-9 UDP中的队列

标识网间通信的进程

- 半相关（half-association）唯一标识一个进程
- 全相关（association）：套接字
 - 网间进程通信由两个进程组成，只能使用同一种高层协议。
 - 两个协议相同的半相关才能组合成一个全相关。



内容纲要

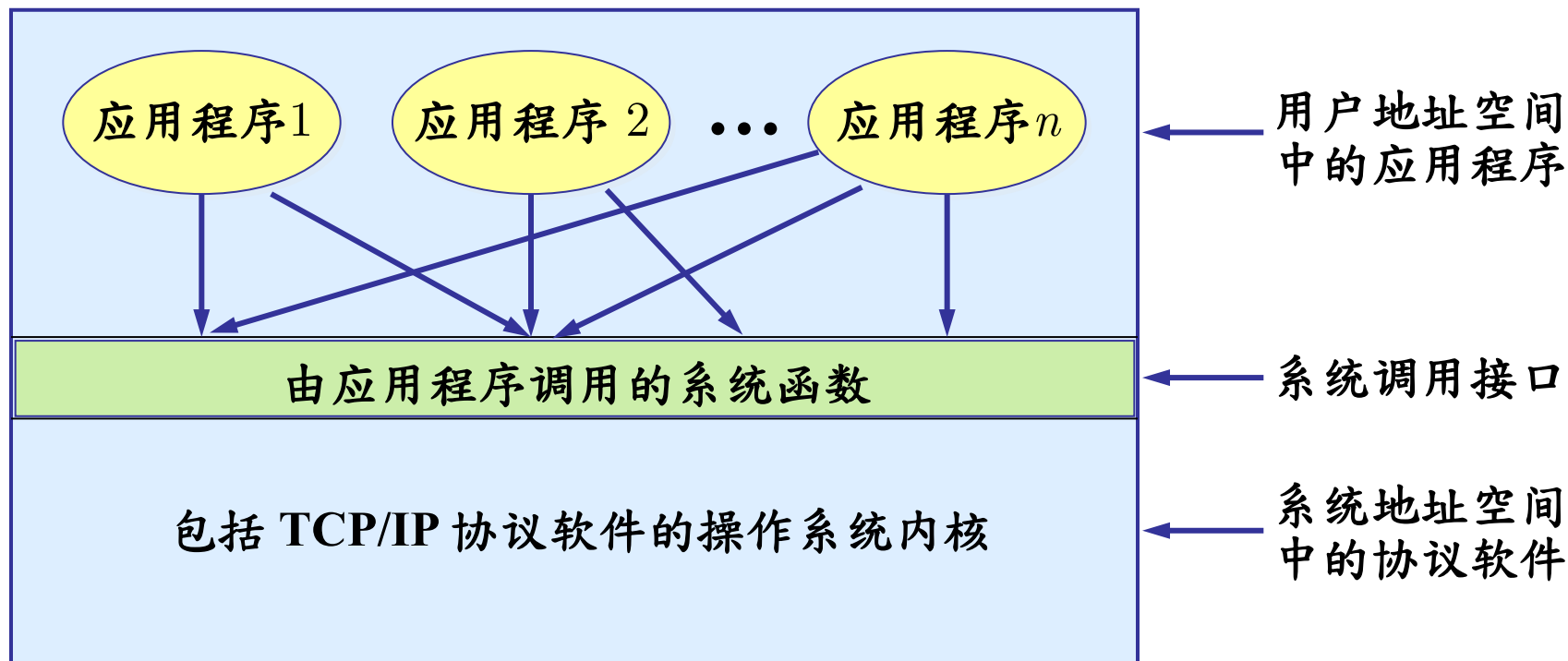
1	客户端和服务端
2	网间进程通信
3	Socket API
4	客户服务器端程序实例
5	小结

套接字 (Socket)

- 套接字是一种操作系统提供的进程间通信机制。
- 套接字是全相关：

本地 端口号	本地 IP地址	协议	远程 IP地址	远程 端口号
-----------	------------	----	------------	-----------
- 套接字描述符是套接字资源的总和的句柄。
 - 应用进程需要使用网络进行通信时，请求操作系统为其创建套接字，把网络通信所需要的系统资源分配给该进程。
 - 通信完毕后，应用进程通过一个关闭套接字的系统调用通知操作系统回收与该“号码”相关的所有资源。
- 操作系统提供Socket API来操作Socket

多个应用进程使用系统调用的机制



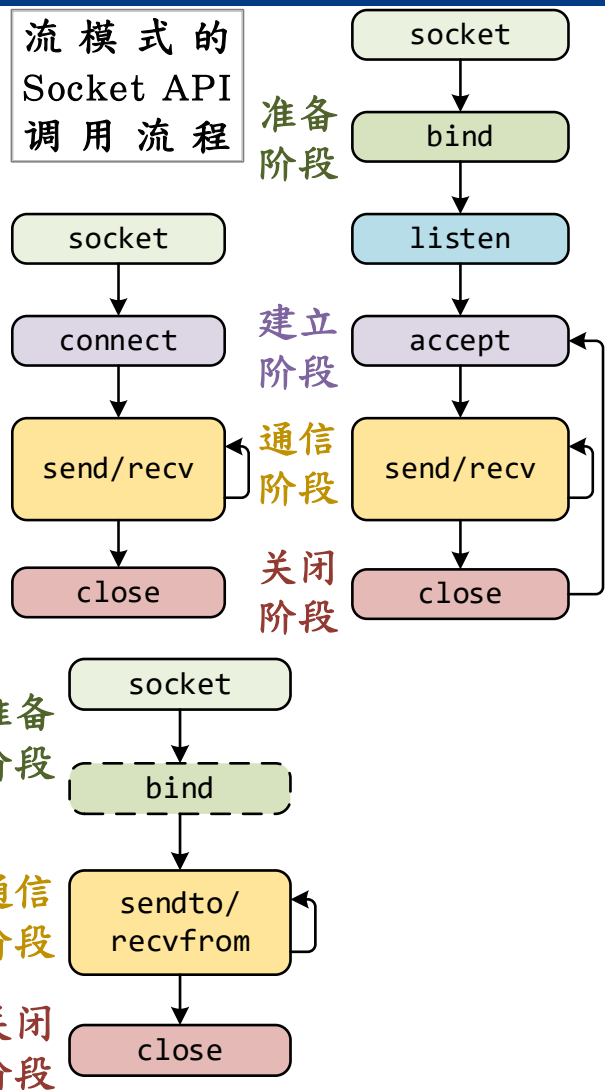
Socket API通信的阶段

• 准备阶段

- ① 创建空的套接字
- ② 把本地地址绑定到套接字
 - TCP必须，UDP可选
- ③ 把套接字设为被动监听模式
 - 随时接受来自客户端的请求。
 - 仅限TCP

• 建立阶段

- ④ 主动向服务器发起连接
- ⑤ 接受来自客户端的连接



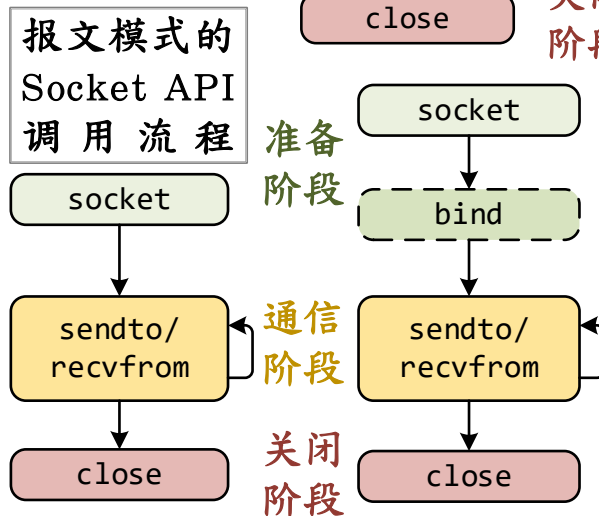
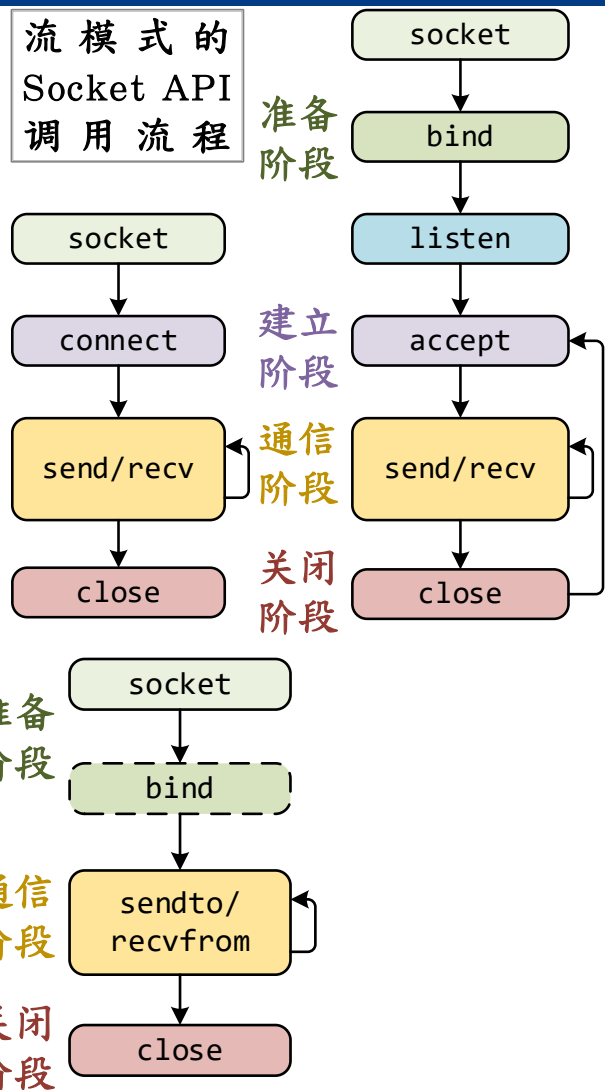
Socket API通信的阶段

- 通信阶段

- 客 服 发送或接收信息，不限次数

- 关闭阶段

- 客 服 关闭连接，释放全部资源



主要的Socket API函数名

函数名	使用方	含义
accept	服务器	接受传入连接
bind	服务器	指定IP地址和协议端口
close	两者均可	终止通信
connect	客户端	连接到远程应用程序
getpeername	服务器	获取客户端的IP地址
getsockopt	服务器	获取套接字的当前选项
listen	服务器	准备套接字供服务器使用
recv	两者均可	接收传入数据或消息
recvmsg	两者均可	接收数据（消息范式）
recvfrom	两者均可	接收消息及发送方地址
send (write)	两者均可	发送传出数据或消息
sendmsg	两者均可	发送传出消息
sendto	两者均可	发送消息（sendmsg的变体）
setsockopt	两者均可	更改套接字选项
shutdown	两者均可	终止连接
socket	两者均可	创建套接字供上述函数使用

编程要求

- Windows Socket编程

- 代码包含头文件Winsock2.h ,
- 编译时链接到库 (导入库) Ws2_32.lib
- 执行时默认路径下存在Ws2_32.dll (主流版本具备)

- UNIX Socket编程

- 代码包含头文件 socket.h
- 已集成在libc库

```
#include <sys/socket.h> // 核心socket函数
#include <netinet/in.h> // 地址结构体
#include <arpa/inet.h> // IP地址转换函数
#include <unistd.h> // close()等文件操作
#include <netdb.h> // 域名解析
```

函数 socket

socket

功能	创建一个用于网络通信的套接字端点。	
格式	SOCKET socket(int <i>af</i> , int <i>type</i> , int <i>protocol</i>);	
参数	<i>af</i>	地址族。
	<i>type</i>	新套接字的类型。
	<i>protocol</i>	要使用的协议。
返回值	成功	新套接字的描述符。
	失败	INVALID_SOCKET
头文件	winsock2.h	
说明	1. 通过调用 WSAGetLastError 来检索特定的错误代码。	

函数 socket

• 参数取值

地址族	
取值	意义
AF_INET	IPv4
AF_INET6	IPv6

套接字的类型

取值	意义
SOCK_STREAM	流模式 (TCP)
SOCK_DGRAM	报文模式 (UDP)
SOCK_RAW	原始模式 (IP)

套接字的类型

取值	意义
IPPROTO_ICMP	ICMP
IPPROTO_TCP	TCP
IPPROTO_UDP	UDP

函数 bind

bind

功能	将本地地址与套接字相关联。	
格式	<pre>int bind(SOCKET s, const sockaddr *addr, int addrlen);</pre>	
参数	<i>s</i>	未绑定套接字的描述符。
	<i>addr</i>	指向本地地址的 <code>sockaddr</code> 结构的指针。
	<i>addrlen</i>	指针 <i>addr</i> 指向的值的长度（字节）。
返回值	成功	0
	失败	SOCKET_ERROR
头文件	winsock2.h	
说明	1. 通过调用 <code>WSAGetLastError</code> 来检索特定的错误代码。	

函数 bind

- 套接字地址结构体（以IPv4为例）
 - sockaddr 结构因所选协议而异，以网络字节顺序表示。

```
struct sockaddr {  
    ushort  sa_family;  
    char    sa_data[14];  
};  
  
struct sockaddr_in {  
    short    sin_family;  
    ushort  sin_port;  
    struct  in_addr sin_addr;  
    char    sin_zero[8];  
};
```

```

// Declare variables
SOCKET ListenSocket;
struct sockaddr_in saServer;
hostent* localHost;
char* localIP;

// Create a listening socket
ListenSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);

// Get the local host information
localHost = gethostbyname("");
localIP = inet_ntoa (*(struct in_addr *)*localHost->h_addr_list);

// Set up the sockaddr structure
saServer.sin_family = AF_INET;
saServer.sin_addr.s_addr = inet_addr(localIP);
saServer.sin_port = htons(5150);

// Bind the listening socket using the
// information in the sockaddr structure
bind( ListenSocket, (SOCKADDR*) &saServer, sizeof(saServer) );

```

网络字节顺序 (htons)

- 字节顺序

- 大端序 (Big-endian) 和小端序 (Little-endian byte order)
- 不同计算机存放多字节值的顺序不同，有的在起始地址存放低位字节 (小数在前)，有的存高位字节 (大数在前)。

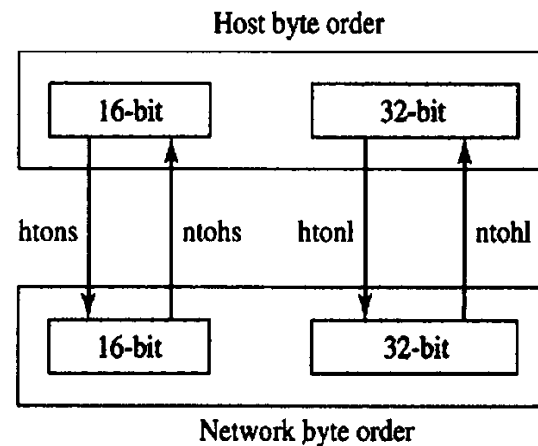
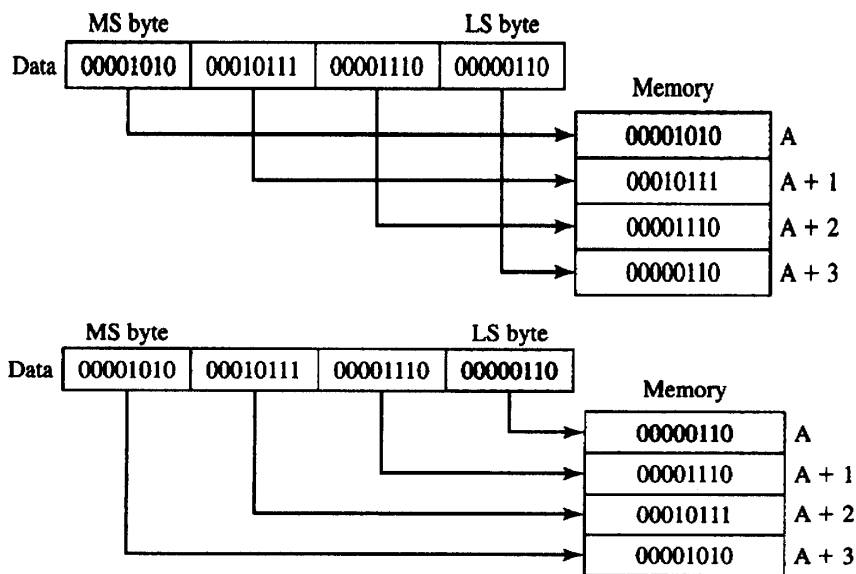
- 网络字节顺序

- 网络协议需指定字节顺序，保证数据的正确性
- TCP/IP使用**16位和32位整数的大数在前**格式

- 主机中的字节顺序

网络字节顺序

- 上图大尾序：高地址为尾
- 下图小尾序：低地址为尾
- 转换： $(host, network)to(h, n)(short, long)$



函数 connect

connect

功能	与指定的套接字建立连接。	
格式	<pre>int connect(SOCKET s, const sockaddr *name, int namelen);</pre>	
参数	<i>s</i>	未连接套接字的描述符。
	<i>addr</i>	指向应建立连接的 sockaddr 结构的指针。
	<i>namelen</i>	指针 <i>addr</i> 指向的值的长度（字节）。
返回值	成功	0
	失败	SOCKET_ERROR
头文件	winsock2.h	
说明	1. 通过调用 WSAGetLastError 来检索特定的错误代码。	

```

// The sockaddr_in structure specifies the address family,
// IP address, and port of the server to be connected to.
sockaddr_in clientService;
clientService.sin_family = AF_INET;
clientService.sin_addr.s_addr = inet_addr("127.0.0.1");
clientService.sin_port = htons(27015);
// Connect to server.
iResult = connect(ConnectSocket, (SOCKADDR *) & clientService,
sizeof (clientService));
if (iResult == SOCKET_ERROR) {
    wprintf(L"connect function failed with error: %ld\n",
WSAGetLastError());
    iResult = closesocket(ConnectSocket);
    if (iResult == SOCKET_ERROR)
        wprintf(L"closesocket function failed with error: %ld\n",
WSAGetLastError());
    WSACleanup();
    return 1;
}
wprintf(L"Connected to server.\n");

```

函数 listen

listen

功能	将套接字置于侦听传入连接的状态。	
格式	<code>int listen(SOCKET s, int backlog);</code>	
参数	<code>s</code>	绑定的未连接的套接字的描述符。
	<code>backlog</code>	挂起的连接队列的最大长度。 如果设置为 SOMAXCONN，则为系统定义的最大连接队列常量。
返回值	成功	0
	失败	SOCKET_ERROR
头文件	winsock2.h	
说明	1. 通过调用 WSAGetLastError 来检索特定的错误代码。	

函数 accept

accept

功能	允许在套接字上进行传入连接尝试。	
格式	SOCKET accept(SOCKET s, sockaddr *addr, int *addrlen);	
参数	s	绑定的未连接的套接字的描述符。
	addr	指向接收连接实体地址的缓冲区的可选指针。
	namelen	指向包含 addr 参数指向的结构长度的可选指针。
返回值	成功	新套接字的描述符。
	失败	INVALID_SOCKET
头文件	winsock2.h	
说明	1. 通过调用 WSAGetLastError 来检索特定的错误代码。 2. 默认为阻塞模式。	


```

// Listen for incoming connection requests.
// on the created socket
if (listen(ListenSocket, 1) == SOCKET_ERROR) {
    wprintf(L"listen failed with error: %ld\n", WSAGetLastError());
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
}
// Create a SOCKET for accepting incoming requests.
SOCKET AcceptSocket;
wprintf(L"Waiting for client to connect...\n");
// Accept the connection.
AcceptSocket = accept(ListenSocket, NULL, NULL);
if (AcceptSocket == INVALID_SOCKET) {
    wprintf(L"accept failed with error: %ld\n", WSAGetLastError());
    closesocket(ListenSocket);
    WSACleanup();
    return 1;
} else
    wprintf(L"Client connected.\n");

```

函数 send

send

功能	在连接的套接字上发送数据。	
格式	<code>int send(SOCKET s, const char *buf, int len, int flags);</code>	
参数	<code>s</code>	已连接套接字的描述符。
	<code>buf</code>	指向包含要传输的数据的缓冲区的指针。
	<code>len</code>	<code>buf</code> 参数指向的缓冲区中数据的长度。
	<code>flags</code>	一组指定调用方式的标志。
返回值	成功	发送的总字节数，该字节数可能小于 <code>len</code> 。
	失败	SOCKET_ERROR
头文件	winsock2.h	

函数 send

- 参数取值

调用方式的标志

取值	意义
MSG_DONTROUTE	数据不经路由，只在本地网络发送。避免发给本地网络的报文经过路由器而浪费时间。但不强制。
MSG_OOB	发送OOB数据，仅限TCP有效。 使用TCP的 URG标志位 和 紧急指针。
上述值取 按位或运算。	

```

// Send an initial buffer
iResult = send( ConnectSocket, sendbuf, (int)strlen(sendbuf), 0 );
if (iResult == SOCKET_ERROR) {
    wprintf(L"send failed with error: %d\n", WSAGetLastError());
    closesocket(ConnectSocket);
    WSACleanup();
    return 1;
}
printf("Bytes Sent: %d\n", iResult);

// shutdown the connection since no more data will be sent
iResult = shutdown(ConnectSocket, SD_SEND);
if (iResult == SOCKET_ERROR) {
    wprintf(L"shutdown failed with error: %d\n", WSAGetLastError());
    closesocket(ConnectSocket);
    WSACleanup();
    return 1;
}

```

函数 recv

recv

功能	从连接的套接字或绑定的无连接套接字接收数据。	
格式	<code>int recv(SOCKET s, char *buf, int len, int flags);</code>	
参数	<code>s</code>	已连接套接字的描述符。
	<code>buf</code>	指向用于接收传入数据的缓冲区的指针。
	<code>len</code>	<code>buf</code> 参数指向的缓冲区中数据的长度。
	<code>flags</code>	一组指定调用方式的标志。
返回值	成功	收到的总字节数。如果连接关闭，返回 0。
	失败	SOCKET_ERROR
头文件	winsock2.h	

函数 recv

• 参数取值

调用方式的标志

取值	意义
MSG_PEEK	查看传入数据。数据将复制到缓冲区中，但不会从输入队列中删除。
MSG_OOB	发送OOB数据，仅限TCP有效。 使用TCP的URG标志位和紧急指针。
MSG_WAITALL	仅当发生以下事件之一时，接收请求才会完成： * 调用方提供的缓冲区已完全满。 * 该连接已关闭。 * 请求已取消或发生错误。 数据报套接字不支持，不能和PEEK、OOB共用。
上述值取 按位或运算。	

```
// Receive until the peer closes the connection
do {

    iResult = recv(ConnectSocket, recvbuf, recvbuflen, 0);
    if ( iResult > 0 )
        wprintf(L"Bytes received: %d\n", iResult);
    else if ( iResult == 0 )
        wprintf(L"Connection closed\n");
    else
        wprintf(L"recv failed with error: %d\n", WSAGetLastError());

} while( iResult > 0 );
```

函数 sendto

sendto

功能	将数据发送到特定目标。	
格式	<pre>int sendto(SOCKET s, const char *buf, int len, int flags, const struct sockaddr *to, int tolen);</pre>	
参数	<i>s</i>	已连接套接字的描述符。
	<i>buf</i>	指向包含要传输的数据的缓冲区的指针。
	<i>len</i>	<i>buf</i> 参数指向的缓冲区中数据的长度。
	<i>flags</i>	一组指定调用方式的标志。
	<i>to</i>	包含目标套接字地址的 <i>sockaddr</i> 结构可选指针。
	<i>tolen</i>	由 <i>to</i> 参数指向地址的数据长度。
返回值	成功	发送的总字节数，该字节数可能小于 <i>len</i> 。
	失败	SOCKET_ERROR
头文件	winsock2.h	

函数 recvfrom

recvfrom

功能	将数据发送到特定目标。	
格式	<pre>int recvfrom(SOCKET s, char *buf, int len, int flags, sockaddr *from, int fromlen);</pre>	
参数	<i>s</i>	已连接套接字的描述符。
	<i>buf</i>	指向传入数据的缓冲区的指针。
	<i>len</i>	<i>buf</i> 参数指向的缓冲区的长度。
	<i>flags</i>	一组指定调用方式的标志。
	<i>from</i>	指向 <i>sockaddr</i> 结构中的缓冲区的可选指针
	<i>fromlen</i>	由 <i>from</i> 参数指向地址的数据长度。
返回值	成功	发送的总字节数，如果已关闭，则返回 0。
	失败	SOCKET_ERROR
头文件	winsock2.h	

```

// Initialize Winsock
iResult = WSASStartup(MAKEWORD(2, 2), &wsaData);
if (iResult != NO_ERROR) { ... }
// Create a socket for sending data
SendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (SendSocket == INVALID_SOCKET) { ... }
// Set up the RecvAddr structure with the IP address of
// the receiver (in this example case "192.168.1.1")
// and the specified port number.
RecvAddr.sin_family = AF_INET;
RecvAddr.sin_port = htons(Port);
RecvAddr.sin_addr.s_addr = inet_addr("192.168.1.1");
// Send a datagram to the receiver
wprintf(L"Sending a datagram to the receiver...\n");
iResult = sendto(SendSocket, SendBuf, BufLen, 0, (SOCKADDR *) &
RecvAddr, sizeof (RecvAddr));
if (iResult == SOCKET_ERROR) { ... }
// When the application is finished sending, close the socket.
wprintf(L"Finished sending. Closing socket.\n");
iResult = closesocket(SendSocket);
if (iResult == SOCKET_ERROR) { ... }

```

```

// Initialize Winsock
iResult = WSASStartup(MAKEWORD(2, 2), &wsaData);
if (iResult != NO_ERROR) { ... }
// Create a socket for sending data
SendSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (SendSocket == INVALID_SOCKET) { ... }
// Set up the RecvAddr structure with the IP address of
// the receiver (in this example case "192.168.1.1")
// and the specified port number.
RecvAddr.sin_family = AF_INET;
RecvAddr.sin_port = htons(Port);
RecvAddr.sin_addr.s_addr = inet_addr("192.168.1.1");
// Send a datagram to the receiver
wprintf(L"Sending a datagram to the receiver...\n");
iResult = sendto(SendSocket, SendBuf, BufLen, 0, (SOCKADDR *) &
RecvAddr, sizeof (RecvAddr));
if (iResult == SOCKET_ERROR) { ... }
// When the application is finished sending, close the socket.
wprintf(L"Finished sending. Closing socket.\n");
iResult = closesocket(SendSocket);
if (iResult == SOCKET_ERROR) { ... }

```

函数 WSASStartup

WSASStartup

功能	将套接字置于侦听传入连接的状态。	
格式	<pre>int WSASStartup(WORD wVersionRequired, LPWSADATA lpWSADATA);</pre>	
参数	<i>wVersionRequired</i>	调用方可以使用的最高版本的 Windows 套接字规范。高位字节指定次要版本号；低位字节指定主版本号。
	<i>lpWSADATA</i>	指向 WSADATA 数据结构的指针，该结构用于接收 Windows 套接字实现的详细信息。
返回值	成功	0
	失败	错误代码
头文件	winsock2.h	
说明	1. 不能使用 WSAGetLastError 来检索特定的错误代码。	

函数 WSStartup

- 假如一个程序要使用2.1版本的Socket，程序代码如下

```
#define MAKEWORD(a, b) (((WORD)(((BYTE)(((DWORD_PTR)(a)) & 0xff)) | \
                                (((WORD)((BYTE)(((DWORD_PTR)(b)) & 0xff))) << 8)))
```

```
wVersionRequested = MAKEWORD( 2, 1 );  
err = WSStartup( wVersionRequested, &wsaData );
```

函数 closesocket

closesocket

功能	关闭现有套接字。	
格式	<code>int closesocket(SOCKET s);</code>	
参数	<code>s</code>	要关闭的套接字的描述符。
返回值	成功	0
	失败	SOCKET_ERROR
头文件	winsock2.h	
说明	1. 通过调用 <code>WSAGetLastError</code> 来检索特定的错误代码。	

内容纲要

1	客户端和服务端
2	网间进程通信
3	Socket API
4	客户服务器端程序实例
5	小结

服务器端编程

```
/* server.c - code for example server program that uses TCP */
#ifdef unix
#define WIN32
#include <windows.h>
#include <winsock.h>
#else
#define closesocket close
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <netdb.h>
#endif

#include <stdio.h>
#include <string.h>

#define PROTOPORT      5193          /* default protocol port number */
#define QLEN           6             /* size of request queue          */

int      visits        =    0;      /* counts client connections    */
```

平台间差异

服务器端编程

```
/*-----  
 * Program:    server  
 *  
 * Purpose:    allocate a socket and then repeatedly execute the following:  
 *              (1) wait for the next connection from a client  
 *              (2) send a short message to the client  
 *              (3) close the connection  
 *              (4) go back to step (1)  
 *  
 * Syntax:     server [ port ]  
 *  
 *              port - protocol port number to use  
 *  
 * Note:       The port argument is optional.  If no port is specified,  
 *              the server uses the default given by PROTOPORT.  
 *-----  
 */  
main(argc, argv)  
int    argc;  
char   *argv[];
```

服务器端编程

```
{  
    struct hostent *ptrh; /* pointer to a host table entry */  
    struct protoent *ptrp; /* pointer to a protocol table entry */  
    struct sockaddr_in sad; /* structure to hold server's address */  
    struct sockaddr_in cad; /* structure to hold client's address */  
    int sd, sd2; /* socket descriptors */  
    int port; /* protocol port number */  
    int alen; /* length of address */  
    char buf[1000]; /* buffer for string the server sends */  
  
#ifdef WIN32  
    WSADATA wsaData;  
    WSAStartup(0x0101, &wsaData);  
#endif  
  
    memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */  
    sad.sin_family = AF_INET; /* set family to Internet */  
    sad.sin_addr.s_addr = INADDR_ANY; /* set the local IP address */  
    /* Check command-line argument for protocol port and extract */  
    /* port number if one is specified. Otherwise, use the default */  
    /* port value given by constant PROTOPORT */
```

WSAStartup应为第一个调用的Windows Socket函数，用于定义版本号。

服务器端编程

```
if (argc>1 && argv[1][0]!='?')
{
    printf("Program:  server\n");
    printf("Purpose:  allocate a socket and then repeatedly execute the
following:\n");
    printf("                (1) wait for the next connection from a client\n");
    printf("                (2) send a short message to the client\n");
    printf("                (3) close the connection\n");
    printf("                (4) go back to step (1)\n");
    printf("Syntax:  server [ port ]\n");
    printf("                port - protocol port number to use\n");
    printf("Note:    The port argument is optional. If no port is
specified,\n");
    printf("                the server uses the default given by PROTOPORT.\n");
    exit(1);
}
```

服务器端编程

```
if (argc > 1) {                                /* if argument specified */
    port = atoi(argv[1]);                       /* convert argument to binary */
} else {
    port = PROTOPORT;                           /* use default port number */
}
if (port > 0)                                  /* test for illegal value */
    sad.sin_port = htons((u_short)port);
else {                                         /* print error message and exit */
    fprintf(stderr, "bad port number %s\n", argv[1]);
    exit(1);
}

/* Map TCP transport protocol name to protocol number */

if ( ((int)(ptrp = getprotobyname("tcp"))) == 0) {
    fprintf(stderr, "cannot map \"tcp\" to protocol number");
    exit(1);
}
```

htons将u_short从主机顺序转换到TCP/IP网络字节顺序（大端）。

Getprotobyname 检索协议名称对应的的协议信息。

服务器端编程

协议族
IPv4

```
/* Create a socket */  
sd = socket(PF_INET, SOCK_STREAM, ptrp->p_proto);
```

新套接字的类型：供了序列、可靠、双向，
基于连接的带OOB数据传输机制的字节流。

```
if (sd < 0) {  
    fprintf(stderr, "socket creation failed\n");  
    exit(1);  
}
```

协议类型

socket函数创建一个绑定到一个
特定运输服务提供商的套接字。

```
/* Bind a local address to the socket */
```

对套接字赋值的
本地地址指针

```
if (bind(sd, (struct sockaddr *)&sad, sizeof(sad)) < 0) {  
    fprintf(stderr, "bind failed\n");  
    exit(1);  
}
```

识别未绑定的套
接字的描述符。

bind函数将本地地址与套接
字相关联。

```
/* Specify size of request queue */
```

```
if (listen(sd, QLEN) < 0) {  
    fprintf(stderr, "listen failed\n");  
    exit(1);  
}
```

队列长度

listen将套接字置于侦听传入
连接的状态之下

服务器端编程

```
/* Main server loop - accept and handle requests */
```

Accept函数允许在套接字上传入的连接尝试。

```
while (1) {  
    alen = sizeof(cad);  
    if ( (sd2=accept(sd, (struct sockaddr *)&cad, &alen)) < 0) {  
        fprintf(stderr, "accept failed\n");  
        exit(1);  
    }  
    visits++;  
    sprintf(buf, "This server has been contacted %d time%s\n",  
            visits, visits==1?".":"s.");  
    send(sd2, buf, strlen(buf), 0);  
    closesocket(sd2);  
}
```

一个死循环

可选的指针，指向到一个缓冲区，它接收连接实体的地址，如已知的通信层。

Send函数向一个已连接的套接字发送数据。

关闭套接字。
注意是哪个套接字

客户端编程

```
/* client.c - code for example client program that uses TCP */

#ifdef unix
#define WIN32
#include <windows.h>
#include <winsock.h>
#else
#define closesocket close
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <netdb.h>
#endif

#include <stdio.h>
#include <string.h>

#define PROTOPORT      5193          /* default protocol port number */
extern int             errno;
char localhost[] = "localhost";    /* default host name */
```

客户端编程

```
/*-----  
 * Program:   client  
 *  
 * Purpose:   allocate a socket, connect to a server, and print all output  
 *  
 * Syntax:    client [ host [port] ]  
 *  
 *            host   - name of a computer on which server is executing  
 *            port   - protocol port number server is using  
 *  
 * Note:      Both arguments are optional.  If no host name is specified,  
 *            the client uses "localhost"; if no protocol port is  
 *            specified, the client uses the default given by PROTOPORT.  
 *-----  
 */  
  
main(argc, argv)  
int     argc;  
char    *argv[];
```


客户端编程

```
{  
    struct hostent *ptrh; /* pointer to a host table entry */  
    struct protoent *ptrp; /* pointer to a protocol table entry */  
    struct sockaddr_in sad; /* structure to hold an IP address */  
    int sd; /* socket descriptor */  
    int port; /* protocol port number */  
    char *host; /* pointer to host name */  
    int n; /* number of characters read */  
    char buf[1000]; /* buffer for data from the server */  
  
#ifdef WIN32  
    WSADATA wsaData;  
    WSAStartup(0x0101, &wsaData);  
#endif  
  
    memset((char *)&sad, 0, sizeof(sad)); /* clear sockaddr structure */  
    sad.sin_family = AF_INET; /* set family to Internet */  
  
    /* Check command-line argument for protocol port and extract */  
    /* port number if one is specified. Otherwise, use the default */  
    /* port value given by constant PROTOPORT */
```

客户端编程

```
if (argc>1 && argv[1][0]!='?')
{
    printf("Program:   client\n");
    printf("Purpose:   allocate a socket, connect to a server, and print all
output\n");
    printf("Syntax:    client [ host [port] ]\n");
    printf("           host - name of a computer on which server is
executing\n");
    printf("           port - protocol port number server is using\n");
    printf("Note:      Both arguments are optional. If no host name is
specified,\n");
    printf("           the client uses \"localhost\"; if no protocol port is\n");
    printf("           specified, the client uses the default given by
PROTOPORT.\n");
    exit(1);
}
```

客户端编程

```
if (argc > 2) {                                /* if protocol port specified */
    port = atoi(argv[2]);                       /* convert to binary */
} else {
    port = PROTOPORT;                          /* use default port number */
}
if (port > 0)                                  /* test for legal value */
    sad.sin_port = htons((u_short)port);
else {                                         /* print error message and exit */
    fprintf(stderr, "bad port number %s\n", argv[2]);
    exit(1);
}

/* Check host argument and assign host name. */

if (argc > 1) {
    host = argv[1];                          /* if host argument specified */
} else {
    host = localhost;
}
```

客户端编程

```
/* Convert host name to equivalent IP address and copy to sad. */
```

```
ptrh = gethostbyname(host);
```

```
if ( ((char *)ptrh) == NULL ) {  
    fprintf(stderr, "invalid host: %s\n", host);  
    exit(1);  
}
```

```
memcpy(&sad.sin_addr, ptrh->h_addr, ptrh->h_length);
```

从主机名获取指向
主机的指针

```
/* Map TCP transport protocol name to protocol number. */
```

```
if ( ((int)(ptrp = getprotobyname("tcp"))) == 0 ) {  
    fprintf(stderr, "cannot map \"tcp\" to protocol number");  
    exit(1);  
}
```

```
/* Create a socket. */
```

```
sd = socket(PF_INET, SOCK_STREAM, ptrp->p_proto);
```

```
if (sd < 0) {  
    fprintf(stderr, "socket creation failed\n");  
    exit(1);  
}
```

客户端编程

```
/* Connect the socket to the specified server. */
```

```
if (connect(sd, (struct sockaddr *)&sad, sizeof(sad)) < 0) {  
    fprintf(stderr, "connect failed\n");  
    exit(1);  
}
```

recv函数从连接的套接字或无连接的套接字绑定接收数据。

```
/* Repeatedly read data from socket and write to user's screen. */
```

```
n = recv(sd, buf, sizeof(buf), 0);  
while (n > 0) {  
    write(1, buf, n);  
    n = recv(sd, buf, sizeof(buf), 0);  
}
```

```
/* Close the socket. */  
closesocket(sd);
```

```
/* Terminate the client program gracefully. */  
exit(0);
```

```
}
```

Microsoft .Net Framework

- System.Net.Sockets 命名空间
- 详见UDP课的PPT



C++实例

```
#using <System.dll>

using namespace System;
using namespace System::Text;
using namespace System::IO;
using namespace System::Net;
using namespace System::Net::Sockets;
String^ DoSocketGet( String^ server )
{
    //Set up variables and String to write to the server.
    Encoding^ ASCII = Encoding::ASCII;
    String^ Get = "GET / HTTP/1.1\r\nHost: ";
    Get->Concat( server, "\r\nConnection: Close\r\n\r\n" );
    array<Byte>^ByteGet = ASCII->GetBytes( Get );
    array<Byte>^RecvBytes = gcnew array<Byte>(256);
    String^ strRetPage = nullptr;

    // IPAddress and IPEndPoint represent the endpoint that will
    // receive the request.
    // Get first IPAddress in list return by DNS.
```

C++实例

```
try
{
    // Define those variables to be evaluated in the next for loop and
    // then used to connect to the server. These variables are defined
    // outside the for loop to make them accessible there after.
    Socket^ s = nullptr;
    IPEndPoint^ hostEndPoint;
    IPAddress^ hostAddress = nullptr;
    int conPort = 80;

    // Get DNS host information.
    IPHostEntry^ hostInfo = Dns::Resolve( server );

    // Get the DNS IP addresses associated with the host.
    array<IPAddress^>^IPaddresses = hostInfo->AddressList;

    // Evaluate the socket and receiving host IPAddress and IPEndPoint.
    for ( int index = 0; index < IPaddresses->Length; index++ )
    {
        hostAddress = IPaddresses[ index ];
    }
}
```


C++实例

```
hostEndPoint = gcnew IPEndPoint( hostAddress,conPort );

// Creates the Socket to send data over a TCP connection.
s = gcnew Socket( AddressFamily::InterNetwork, SocketType::Stream,
ProtocolType::Tcp );

// Connect to the host using its IPEndPoint.
s->Connect( hostEndPoint );
if ( !s->Connected )
{
    // Connection failed, try next IPaddress.
    strRetPage = "Unable to connect to host";
    s = nullptr;
    continue;
}

// Sent the GET request to the host.
s->Send( ByteGet, ByteGet->Length, SocketFlags::None );
}
```

C++实例

```
// Receive the host home page content and loop until all the data is received.
Int32 bytes = s->Receive( RecvBytes, RecvBytes->Length, SocketFlags::None );
strRetPage = "Default HTML page on ";
strRetPage->Concat( server, ":\r\n", ASCII->GetString( RecvBytes, 0, bytes ) );
while ( bytes > 0 )
{
    bytes = s->Receive( RecvBytes, RecvBytes->Length, SocketFlags::None );
    strRetPage->Concat( ASCII->GetString( RecvBytes, 0, bytes ) );
}
}
catch ( SocketException^ e )
{
    Console::WriteLine( "SocketException caught!!!" );
    Console::Write( "Source : " );
    Console::WriteLine( e->Source );
    Console::Write( "Message : " );
    Console::WriteLine( e->Message );
}
```

C++实例

```
catch ( ArgumentException^ e )
{
    Console::WriteLine( "ArgumentNullException caught!!!" );
    Console::Write( "Source : " );
    Console::WriteLine( e->Source );
    Console::Write( "Message : " );
    Console::WriteLine( e->Message );
}
catch ( NullReferenceException^ e )
{
    Console::WriteLine( "NullReferenceException caught!!!" );
    Console::Write( "Source : " );
    Console::WriteLine( e->Source );
    Console::Write( "Message : " );
    Console::WriteLine( e->Message );
}
```

C++实例

```
catch ( Exception^ e )
{
    Console::WriteLine( "Exception caught!!!" );
    Console::Write( "Source : " );
    Console::WriteLine( e->Source );
    Console::Write( "Message : " );
    Console::WriteLine( e->Message );
}

return strRetPage;
}

int main()
{
    Console::WriteLine( DoSocketGet( "localhost" ) );
}
```

C++实例：输出

Default HTML page on 127.0.0.1:

HTTP/1.1 200 OK

Server: ASP.NET Development Server/11.0.0.0

Date: Mon, 29 Apr 2013 09:54:33 GMT

X-AspNet-Version: 4.0.30319

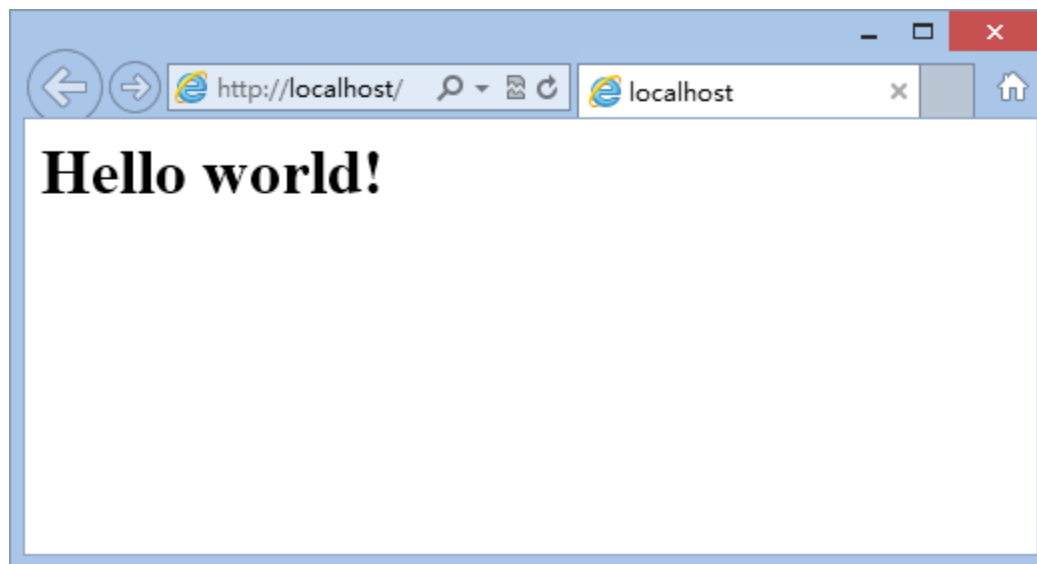
Cache-Control: private

Content-Type: text/html; charset=utf-8

Content-Length: 25

Connection: Close

`<h1>Hello world!</h1>`



谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang